个人学术网站维护手册

一、手册用途

本手册用于说明个人学术网站的日常维护方法，适用于以下场景：

- 自己后续持续添加论文、工具与资料
- 同学或后续维护者接手网站更新
- 使用 VS Code 进行本地测试与修改
- 将网站代码上传到 GitHub 进行备份或部署

本网站当前采用 **React + Vite + 分文件结构** 的方式搭建。整体思路是：

- `data` 文件夹负责存放内容数据
- `components` 文件夹负责存放页面组件
- `styles` 文件夹负责存放样式配置
- `App.jsx` 负责组合整个首页

因此，后续维护时通常只需要修改数据文件，而不需要频繁改动核心页面逻辑。

二、项目结构说明

以下是当前网站建议使用的基础结构：

```
src/
├── App.jsx
├── data/
│   ├── content.js
│   ├── tools.js
│   ├── papers.js
│   ├── societies.js
│   ├── study.js
│   └── templates.js
├── components/
│   ├── Sidebar.jsx
│   ├── Tag.jsx
│   ├── CardLink.jsx
│   ├── PaperRow.jsx
│   └── SectionTable.jsx
└── styles/
    └── appStyles.js
```

三、每个文件的作用

1. `src/App.jsx`

这是网站首页的主入口文件，相当于整个页面的总控制台。

它的主要作用包括：

- 导入所有数据文件
- 导入所有组件
- 控制语言切换（中文 / 韩语）
- 控制论文搜索与分类筛选
- 组合首页的各个版块，例如：
- 首页介绍区
- 论文索引区
- 学术工具区
- 考试资料区
- 模板下载区
- 网站说明区

2. `src/data/content.js`

这个文件负责存放网站界面的文字内容，主要是双语文本。

例如：

- 网站标题
- 按钮文字
- “论文索引”“工具导航”等栏目名称
- 首页说明文字
- 中韩双语切换内容

适合修改的内容

如果你想修改以下内容，就改这个文件：

- 网站名称
- 首页介绍文字
- 按钮文字
- 中文版 / 韩文版页面文案

---

3. `src/data/tools.js`

这个文件负责存放网站中的学术工具、数据库、图书馆链接。

例如：

- ChatGPT
- KCI

- RISS
- 中国知网
- 岭南大学图书馆
- 中国哲学书电子化计划

每条数据一般包含：

- `name`：名称
- `url`：链接地址
- `tag`：标签（如 AI / KR / CN）
- `descZh`：中文说明
- `descKo`：韩文说明

适合修改的内容

如果你想：

- 新增一个数据库链接
- 删除失效工具
- 修改工具简介

就改这个文件。

---

4. `src/data/papers.js`

这是最重要的数据文件之一，用于存放论文索引。

每篇论文一般包含以下字段：

- `title`：论文题目
- `author`：作者
- `category`：分类（中文和韩文）
- `year`：年份
- `link`：论文链接
- `source`：来源（可选，例如 KCI / CNKI）

适合修改的内容

如果你想：

- 添加新论文
- 删除旧论文
- 修改论文分类
- 更正年份、作者、链接

就改这个文件。

后面“如何更新论文资料”部分会详细说明具体写法。

---

5. `src/data/societies.js`

这个文件用于存放相关学会、学术期刊、投稿入口等信息。

例如：

- 영남중국어문학회
- 동아인문학회
- JAMS 投稿系统

每条数据一般包含：

- `name`：学会名称
- `url`：官网或投稿入口
- `noteZh`：中文说明
- `noteKo`：韩文说明

适合修改的内容

如果你想：

- 增加投稿网站入口
- 增加学会介绍
- 调整学会顺序

就改这个文件。

---

6. `src/data/study.js`

这个文件用于存放考试资料、课程资料、专题复习资料等资源。

例如：

- 毕业考试资料
- 课程复习压缩包
- 百度网盘 / Google Drive 链接

每条数据一般包含：

- `name`：资料名称
- `note`：说明文字
- `url`：下载地址

适合修改的内容

如果你想：

- 上传新的考试资料链接
- 修改资料说明
- 更换下载地址

就改这个文件。

---

7. `src/data/templates.js`

这个文件用于存放论文模板与投稿入口。

例如：

- 硕士论文模板
- 博士论文模板
- 学会投稿规范
- Hancom 官方下载链接

每条数据一般包含：

- `type`：类型名称
- `note`：说明文字
- `url`：链接

适合修改的内容

如果你想：

- 补充论文模板
- 替换模板下载地址
- 增加投稿须知链接

就改这个文件。

---

8. `src/components/Sidebar.jsx`

这个文件负责左侧侧边导航栏。

主要功能：

- 显示网站标题
- 显示栏目导航
- 提供语言切换按钮

如果你想改：

- 左侧导航顺序
- 左侧按钮文字逻辑
- 侧栏外观

就修改这个文件。

---

9. `src/components/Tag.jsx`

这个文件负责分类筛选按钮，例如：

- 全部
- 文学
- 语言学
- 文献学
- NLP / AI语言学

如果你想改：

- 分类按钮颜色
- 点击后的高亮样式
- 按钮圆角与大小

就修改这个文件。

---

10. `src/components/CardLink.jsx`

这个文件负责显示工具区的卡片。

例如每个数据库、AI 工具、学术网站入口，都会以卡片形式显示。

如果你想改：

- 工具卡片样式
- 工具说明的排版
- 卡片 hover 效果

就修改这个文件。

---

11. `src/components/PaperRow.jsx`

这个文件负责论文表格中的每一行。

每一篇论文在表格中如何显示，主要由这个组件决定。

如果你想改：

- 论文标题显示方式
- 作者、年份、来源列样式
- “查看”按钮样式

就修改这个文件。

---

12. `src/components/SectionTable.jsx`

这个文件负责论文区域的大表格容器。

包括：

- 表头
- 表格外边框
- 横向滚动
- 基础表格排版

如果你想改：

- 表格表头样式
- 表格整体边框
- 表格背景风格

就修改这个文件。

---

13. `src/styles/appStyles.js`

这个文件负责整站通用样式类名配置。

主要用于统一：

- 页面背景
- 主内容宽度
- 卡片圆角
- Hero 区样式
- 搜索框样式
- 按钮样式

如果你想整体调整网站风格，例如：

- 更学术
- 更极简
- 更浅色
- 更像 KCI / Notion

优先修改这个文件。

四、如何更新论文资料

这是日常维护中最常见、最重要的操作。

1. 打开文件

进入：

```
src/data/papers.js
```

这个文件中每一条论文数据一般写成如下格式：

```
{
  title: "论文题目",
  author: "作者姓名",
  category: { zh: "文学", ko: "문학" },
  year: 2025,
  link: "论文链接",
  source: "KCI"
}
```

2. 新增论文的方法

直接在数组中复制一条旧数据，修改内容即可。

例如新增一篇论文：

```
{
  title: "中国古典小说中的空间叙事研究",
  author: "张三 · 박명진",
  category: { zh: "文学", ko: "문학" },
  year: 2026,
  link: "https://example.com/paper1",
  source: "KCI"
}
```

3. 注意事项

（1）每条论文之间要用逗号隔开

错误示例：

```
{ ... }
{ ... }
```

正确示例：

```
{ ... },
{ ... },
```

（2）最后一个对象后面可以不加逗号

建议统一风格，方便维护。

（3）分类名称必须与 `content.js` 中的分类一致

例如中文分类通常使用：

- 文学
- 语言学
- 文字学
- 文献学
- NLP / AI语言学
- 文化콘텐츠

如果分类写错，页面筛选时可能无法正确显示。

（4）年份尽量使用数字

正确：

```
year: 2025
```

尽量不要写成：

```
year: "2025"
```

虽然页面很多时候也能显示，但排序时使用数字更稳妥。

（5）链接必须完整

正确：

```
link: "https://www.kci.go.kr/..."
```

不要漏掉 `https://`。

---

4. 修改现有论文

只要找到对应对象，直接改字段即可。

例如修改年份：

```
year: 2024
```

例如修改分类：

```
category: { zh: "文化콘텐츠", ko: "문화콘텐츠" }
```

---

5. 删除论文

直接删除对应那一整条对象即可。

删除时要注意数组中的逗号是否正确，否则页面可能报错。

五、如何更新工具、模板、资料

原理与论文更新类似。

1. 更新工具

修改文件：

```
src/data/tools.js
```

新增时按照类似格式：

```
{
  name: "新工具名称",
  url: "https://example.com",
  tag: "AI",
  descZh: "中文说明",
  descKo: "韩文说明"
}
```

2. 更新考试资料

修改文件：

```
src/data/study.js
```

若下载地址变了，就修改：

```
url: "/files/graduate-exam-materials.zip"
```

如果你要换成百度网盘、Google Drive、Dropbox 等链接，也可以直接替换成完整网址。

3. 更新模板下载

修改文件：

```
src/data/templates.js
```

例如新增期刊模板：

```
{
  type: { zh: "某期刊投稿模板", ko: "모 학술지 투고 템플릿" },
  note: { zh: "官方模板下载", ko: "공식 템플릿 다운로드" },
  url: "https://example.com/template"
}
```

六、如何用 VS Code 本地测试

1. 准备环境

在开始前，需要先安装：

- **VS Code**
- **Node.js**（建议安装 LTS 版本）
- **npm**（一般安装 Node.js 后自动带有）

安装完成后，可以在终端输入：

```
node -v
npm -v
```

如果能显示版本号，说明安装成功。

---

2. 用 VS Code 打开项目

操作步骤：

1. 打开 VS Code
2. 点击 `文件` → `打开文件夹`
3. 选择你的网站项目文件夹

例如项目文件夹可能是：

```
D:\my-academic-blog
```

---

3. 安装依赖

第一次打开项目时，需要在 VS Code 终端中安装依赖。

打开终端的方法：

- 菜单栏点击 `终端` → `新建终端`
- 或快捷键 `Ctrl + ~`

然后输入：

```
npm install
```

作用：

- 安装 React、Vite、lucide-react、Tailwind 等项目依赖

如果项目之前已经安装过依赖，一般不用反复执行。

---

4. 启动本地开发服务器

在终端输入：

```
npm run dev
```

通常会出现类似输出：

```
VITE v5.x.x ready in ...
Local:  http://localhost:5173/
```

然后：

1. 按住 `Ctrl`
2. 点击终端中的本地网址
3. 浏览器会打开网站预览页

---

5. 修改后自动刷新

Vite 的优点是：

- 修改代码后
- 保存文件（Ctrl + S）
- 浏览器通常会自动刷新

因此你可以边改边看效果。

---

6. 常见报错处理

（1）页面白屏

优先检查：

- 某个文件导入路径写错
- `tools.js`、`papers.js` 中是否漏了逗号
- 组件文件名与导入名大小写是否一致

（2）终端报错 `Failed to resolve import`

说明某个文件路径不对。

例如：

```
import { layout } from "./styles/appStyles";
```

需要确认：

- `src/styles/appStyles.js` 是否存在
- 文件名大小写是否一致

（3）终端报错 `Unexpected token`

通常说明：

- 少了逗号
- 少了括号
- 多写了引号
- 数组或对象格式不完整

这种情况最常见于 `papers.js` 维护时。

七、如何上传到 GitHub

GitHub 的主要作用有两个：

- 代码备份
- 未来部署网页

---

1. 注册并登录 GitHub

先准备一个 GitHub 账号。

---

2. 创建仓库（Repository）

登录 GitHub 后：

1. 点击右上角 `+`
2. 选择 `New repository`
3. 输入仓库名，例如：

```
my-academic-portal
```

1. 选择公开（Public）或私有（Private）
2. 点击 `Create repository`

---

3. 在本地项目中初始化 Git

在 VS Code 终端中进入项目根目录后，输入：

```
git init
```

然后输入：

```
git add .
```

作用：

- 将当前项目所有文件加入暂存区

接着输入：

```
git commit -m "Initial commit"
```

作用：

- 创建第一次提交记录

---

4. 关联 GitHub 仓库

GitHub 创建新仓库后，会显示一个仓库地址，例如：

```
https://github.com/你的用户名/my-academic-portal.git
```

在终端中输入：

```
git branch -M main
git remote add origin https://github.com/你的用户名/my-academic-portal.git
git push -u origin main
```

第一次上传后，后续只需要：

```
git add .
git commit -m "更新论文与工具数据"
git push
```

八、如何通过 GitHub 更新网站

当你修改了网站内容后，标准流程如下：

```
git add .
git commit -m "更新某某内容"
git push
```

例如：

```
git add .
git commit -m "新增2026年文学论文"
git push
```

建议每次更新都写清楚提交说明，便于以后查看历史记录。

九、如何部署网站（基础说明）

如果只是保存代码，上传到 GitHub 就够了。

如果你想让别人通过网址访问网站，可以考虑部署到：

- GitHub Pages
- Vercel
- Netlify

其中对于 React + Vite 项目，**Vercel** 通常最简单。

Vercel 基本流程

1. 登录 Vercel
2. 绑定 GitHub
3. 选择你的仓库

4. 导入项目
5. 自动部署

之后每次你向 GitHub push 新代码，Vercel 通常会自动重新部署。

十、建议的日常维护流程

建议按照以下顺序维护网站：

情况一：新增论文

1. 打开 `src/data/papers.js`
2. 增加论文对象
3. 保存文件
4. 执行 `npm run dev` 检查页面
5. 确认无误后执行：

```
git add .
git commit -m "新增论文资料"
git push
```

情况二：新增工具网站

1. 打开 `src/data/tools.js`
2. 增加工具对象
3. 本地检查
4. push 到 GitHub

情况三：修改页面文案

1. 打开 `src/data/content.js`
2. 修改中文或韩文文本
3. 检查页面显示
4. push 到 GitHub

情况四：改视觉风格

1. 打开 `src/styles/appStyles.js`
2. 统一调整圆角、背景、边框、按钮样式
3. 本地检查页面风格一致性
4. 再提交到 GitHub

十一、维护时最容易出错的地方

1. 数据格式错误

最常见。

例如：

- 少逗号
- 少括号
- 引号没闭合
- 对象字段写错

2. 文件路径错误

例如：

- `./components/Sidebar`
- `./styles/appStyles`

如果路径写错，页面会直接报错。

3. 分类不一致

例如论文数据里写成：

```
category: { zh: "文學", ko: "문학" }
```

但系统分类是：

```
"文学"
```

这样筛选时可能出问题。

4. 链接失效

建议定期检查：

- KCI 论文链接是否还能打开
- 网盘资料是否失效
- 学会官网是否变更

十二、建议的维护原则

为了让网站长期可用，建议遵守以下原则：

1. 数据与样式分开改

- 改内容：优先改 `data`
- 改外观：优先改 `styles` 和 `components`

2. 尽量不要直接在 `App.jsx` 堆太多新逻辑

`App.jsx` 负责整体组合，不适合堆太多细节。

3. 保持命名统一

例如：

- 论文分类保持统一
- 中韩双语键名保持统一
- 文件名与组件名保持一致

4. 每次修改后先本地测试

不要改完就直接上传 GitHub，先运行：

```
npm run dev
```

确认页面正常显示后再提交。

十三、建议的后续扩展方向

以后网站还可以继续扩展为：

- 教授论文专区
- 同学论文专区
- 课程资料专区
- 学术会议记录
- 投稿记录与审稿记录
- 常用韩语学术表达整理
- 中文 / 韩文摘要模板区

如果继续扩展，建议新增新的数据文件和组件，而不是把所有内容都继续堆进 `App.jsx`。

十四、总结

本网站的维护逻辑可以概括为一句话：

内容更新主要改 `data`，页面结构主要看 `App.jsx`，局部显示看 `components`，整体风格看 `styles`。

对于日常使用者而言，最常用的维护文件通常只有这几个：

- `src/data/papers.js`
- `src/data/tools.js`
- `src/data/content.js`
- `src/data/templates.js`
- `src/data/study.js`

只要掌握这几个文件的修改方法，就已经可以完成大部分网站维护工作。

如果后续网站规模扩大，再逐步增加新的组件和子页面即可。